LAPORAN PRATIKUM STRUKTUR DATA

“ QUEUE ARRAY”

OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : Dr. WAHYUDI, S.T, M.T
ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI INFORMASI DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data queue serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB II PEMBAHASAN 2
2.1 Dasar Teori 2
2.2 Penjelasan Kode Program 3
2.2.1 Class QueueArray_2511531007 3
2.2.2 Class QueueArrayDriver_2511531007 4
2.2.3 Class ReverseData_2511531007 5
2.2.5 Class QueueLinkedList_2511531007 7
2.3 Tugas Praktikum 8
2.3.1 Class AntrianLoket_2511531007 8
BAB III PENUTUP 11
3.1 Kesimpulan 11
3.2 Saran 11
DAFTAR PUSTAKA 12

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang berperan penting dalam pengelolaan dan penyimpanan data secara efisien. Salah satu struktur data yang sering digunakan adalah queue (antrian), yaitu struktur data yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini banyak dijumpai dalam kehidupan sehari-hari, seperti antrian di loket pelayanan, kasir, maupun sistem antrean dalam dunia komputer.

Dalam pemrograman Java, queue dapat diimplementasikan dengan memanfaatkan library seperti LinkedList maupun dengan cara manual menggunakan array. Penggunaan library mempermudah proses implementasi, sedangkan penggunaan array membantu dalam memahami cara kerja queue secara lebih mendalam, seperti pengelolaan posisi depan (front) dan belakang (rear). Oleh karena itu, melalui praktikum dan tugas ini, mahasiswa diharapkan dapat memahami konsep queue secara teori maupun praktik, serta mampu mengimplementasikannya dalam bentuk program sederhana.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dasar struktur data queue (FIFO / First In First Out)
2. Mampu mengimplementasikan queue menggunakan Java
3. Memahami operasi dasar queue seperti enqueue, dequeue, dan display
4. Membedakan penggunaan queue dengan library (LinkedList) dan implementasi manual (array)

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Memahami konsep dasar struktur data queue (FIFO / First In First Out)
2. Mampu mengimplementasikan queue menggunakan Java
3. Memahami operasi dasar queue seperti enqueue, dequeue, dan display
4. Membedakan penggunaan queue dengan library (LinkedList) dan implementasi manual (array).

BAB II
PEMBAHASAN

2.1 Dasar Teori

Queue merupakan salah satu struktur data linear yang menerapkan prinsip FIFO (First In First Out), yaitu elemen yang pertama kali masuk akan menjadi elemen pertama yang keluar. Struktur ini sering digunakan untuk mengatur data yang diproses secara berurutan, seperti pada sistem antrian. Dalam queue terdapat dua operasi utama, yaitu *enqueue* untuk menambahkan data ke bagian belakang antrian dan *dequeue* untuk menghapus data dari bagian depan antrian. Selain itu, terdapat operasi pendukung seperti *peek/front* untuk melihat elemen terdepan tanpa menghapusnya, serta pengecekan kondisi apakah antrian dalam keadaan kosong atau penuh.

Dalam bahasa pemrograman Java, queue dapat diimplementasikan dengan dua cara, yaitu menggunakan library bawaan seperti LinkedList dan dengan cara manual menggunakan array. Penggunaan LinkedList memudahkan dalam pengelolaan data karena tidak perlu mengatur indeks secara langsung. Sementara itu, implementasi menggunakan array membutuhkan pengelolaan variabel seperti *front* dan *rear* untuk menentukan posisi elemen. Pada beberapa kasus, digunakan konsep *circular queue* agar penggunaan kapasitas array lebih efisien, sehingga indeks dapat kembali ke awal ketika mencapai batas akhir array. Dengan memahami kedua pendekatan ini, konsep queue dapat diterapkan secara lebih fleksibel sesuai kebutuhan program.

2.2 Penjelasan Kode Program

2.2.1 Class QueueArray_2511531007

```java
package pekan4_2511531007;

public class QueueArray_2511531007 {
    int front_1007, rear_1007, size_1007;
    int capacity_1007;
    int array_1007[];

    public QueueArray_2511531007(int capacity_1007) {
        this.capacity_1007 = capacity_1007;
        front_1007 = this.size_1007 = 0;
        rear_1007 = capacity_1007 - 1;
        array_1007 = new int[this.capacity_1007];
    }

    boolean isFull_1007(QueueArray_2511531007 queue_1007) {
        return (queue_1007.size_1007 == queue_1007.capacity_1007);
    }

    boolean isEmpty_1007(QueueArray_2511531007 queue_1007) {
        return (queue_1007.size_1007 == 0);
    }

    void enqueue_1007(int item_1007) {
        if (isFull_1007(this))
            return;
        this.rear_1007 = (this.rear_1007 + 1) % this.capacity_1007;
        this.array_1007[this.rear_1007] = item_1007;
        this.size_1007 = this.size_1007 + 1;
        System.out.println(item_1007 + " enqueued to queue");
    }
    int dequeue_1007() {
        if (isEmpty_1007(this))
            return Integer.MIN_VALUE;
        int item_1007 = this.array_1007[this.front_1007];
        this.front_1007 = (this.front_1007 + 1) % this.capacity_1007;
        this.size_1007 = this.size_1007 - 1;
        return item_1007;
    }

    int front_1007() {
        if (isEmpty_1007(this))
            return Integer.MIN_VALUE;

        return this.array_1007[this.front_1007];
    }

    int rear_1007() {
        if (isEmpty_1007(this))
            return Integer.MIN_VALUE;
        return this.array_1007[this.rear_1007];
    }

    // mencetak elemen antrian
    void display_1007() {
        int i_1007;
        if (front_1007 == rear_1007) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        // kunjungi dari belakang dan cetak
        for (i_1007 = front_1007; i_1007 < rear_1007; i_1007++) {
            System.out.printf(" %d <-- ", array_1007[i_1007]);
        }

        return;
    }

}
```

Program ini bikin struktur data queue (antrian**)** pakai array, tapi versi yang agak lebih canggih yaitu circular queue. Jadi data yang masuk itu tetap pakai aturan FIFO, yang masuk duluan keluar duluan. Di dalam class ini ada beberapa variabel penting. front_1007 itu nunjuk ke data paling depan (yang bakal keluar duluan), rear_1007 itu nunjuk ke posisi terakhir (tempat masuk data baru), size_1007 buat ngitung jumlah isi antrian, dan capacity_1007 itu batas maksimalnya. Terus array_1007[] dipakai buat nyimpen semua data antrian.

Di constructor, semua nilai awal disiapin. front mulai dari 0, size juga 0 karena masih kosong, dan rear diset ke capacity - 1. Ini kelihatan aneh, tapi sebenarnya biar pas pertama kali nambah data, posisi rear langsung pindah ke index 0. Nah di sini

keliatan kalau antriannya pakai sistem muter (circular), jadi kalau sudah sampai ujung array, dia balik lagi ke depan.

Method isFull_1007() dan isEmpty_1007() dipakai buat ngecek kondisi antrian. Kalau size sudah sama dengan kapasitas, berarti penuh. Kalau size masih 0, berarti kosong. Ini penting supaya program gak asal nambah atau hapus data yang bisa bikin error.

Terus di enqueue_1007(), itu buat nambah data ke antrian. Dia bakal cek dulu apakah penuh atau tidak. Kalau belum penuh, rear digeser pakai operasi modulo biar tetap muter di dalam batas array, terus datanya dimasukin, dan size ditambah. Kebalikannya, dequeue_1007() dipakai buat ngambil data dari depan. Kalau kosong, dia balikin nilai khusus. Kalau ada isi, dia ambil data di front, terus front digeser, dan size dikurangin.

Ada juga method front_1007() dan rear_1007() buat lihat isi depan dan belakang tanpa ngapus datanya, jadi cuma ngecek doang. Terakhir, display_1007() dipakai buat nampilin isi antrian, walaupun masih sederhana karena belum sepenuhnya menyesuaikan dengan sistem circular (jadi belum sempurna banget kalau datanya sudah muter).

2.2.2 Class QueueArrayDriver_2511531007

```java
package pekan4_2511531007;

public class QueueArrayDriver_2511531007 {

    public static void main(String[] args) {
        QueueArray_2511531007 queue_1007 = new QueueArray_2511531007(1000);
        queue_1007.enqueue_1007(10);
        queue_1007.enqueue_1007(20);
        queue_1007.enqueue_1007(30);
        queue_1007.enqueue_1007(40);
        System.out.println("Item di depan " + queue_1007.front_1007());
        System.out.println("Item paling belakang " + queue_1007.rear_1007());
        System.out.println("tampilan queue");
        queue_1007.display_1007();
        System.out.println();
        System.out.println(queue_1007.dequeue_1007() + " dihapus dari queue");
        System.out.println("Item di depan: " + queue_1007.front_1007());
        System.out.println("Item di belakang: " + queue_1007.rear_1007());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue_1007.display_1007();
    }

}
```

Program pada class QueueArrayDriver_2511531007 berfungsi sebagai program utama untuk menjalankan dan menguji operasi pada struktur data queue yang telah dibuat sebelumnya. Pada bagian awal, dibuat sebuah objek queue_1007 dari class QueueArray_2511531007 dengan kapasitas 1000, yang berarti antrian dapat menampung hingga 1000 data. Selanjutnya, beberapa data yaitu 10, 20, 30, dan 40

dimasukkan ke dalam antrian menggunakan method enqueue_1007(), di mana data ditambahkan secara berurutan ke bagian belakang antrian.

Setelah data dimasukkan, program menampilkan elemen yang berada di posisi depan dan belakang antrian menggunakan method front_1007() dan rear_1007(). Hal ini bertujuan untuk mengetahui elemen yang akan keluar terlebih dahulu serta elemen yang terakhir masuk ke dalam antrian. Kemudian, method display_1007() dipanggil untuk menampilkan seluruh isi antrian sehingga urutan data dapat dilihat secara langsung.

Selanjutnya, dilakukan proses dequeue_1007() untuk menghapus satu elemen dari bagian depan antrian. Elemen yang dihapus adalah data yang pertama kali dimasukkan, sesuai dengan prinsip FIFO (First In First Out). Setelah proses penghapusan, program kembali menampilkan elemen terdepan dan terbelakang yang baru, serta menampilkan isi antrian yang telah diperbarui. Dengan demikian, program ini menunjukkan bagaimana operasi dasar queue seperti penambahan, penghapusan, dan penampilan data dapat dilakukan dan diamati hasilnya.

2.2.3 Class ReverseData_2511531007

```java
package pekan4_2511531007;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511531007 {

    public static void main(String[] args) {
        Queue<Integer> q_1007 = new LinkedList<Integer>();
        q_1007.add(1);
        q_1007.add(2);
        q_1007.add(3); // [1, 2, 3]
        System.out.println("sebelum reverse = " + q_1007);
        Stack<Integer> s_1007 = new Stack<Integer>();
        while (!q_1007.isEmpty()) { // Q -> S
            s_1007.push(q_1007.remove());
        }
        while (!s_1007.isEmpty()) { // S -> Q
            q_1007.add(s_1007.pop());
        }
        System.out.println("sesudah reverse = " + q_1007); // [3, 2, 1]
    }

}
```

Program pada class ReverseData_2511531007 digunakan untuk menunjukkan cara membalik urutan data dalam sebuah queue dengan bantuan stack. Pada awal program, dibuat sebuah objek queue q_1007 menggunakan LinkedList, lalu dimasukkan beberapa data yaitu 1, 2, dan 3 sehingga urutannya menjadi [1, 2, 3]. Setelah itu, isi queue ditampilkan untuk menunjukkan kondisi sebelum dilakukan proses pembalikan.

Selanjutnya, dibuat sebuah stack s_1007 yang akan digunakan sebagai alat bantu. Proses pembalikan dilakukan dalam dua tahap. Tahap pertama adalah memindahkan semua elemen dari queue ke stack menggunakan perulangan while. Karena queue bersifat FIFO dan stack bersifat LIFO, maka saat elemen dipindahkan ke stack, urutannya akan otomatis terbalik. Tahap kedua adalah memindahkan kembali elemen dari stack ke queue. Ketika data diambil dari stack menggunakan pop() dan dimasukkan kembali ke queue, urutan yang sudah terbalik tersebut akan tersimpan di dalam queue.

Pada akhir program, isi queue ditampilkan kembali untuk menunjukkan hasil setelah proses reverse, yaitu menjadi [3, 2, 1].

2.2.4 Class IterasiQueue_2511531007

```java
package pekan4_2511531007;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511531007 {

    public static void main(String[] args) {
        Queue<String> q_1007 = new LinkedList<>();

        q_1007.add("Praktikum");
        q_1007.add("Struktur");
        q_1007.add("Data");
        q_1007.add("Dan");
        q_1007.add("Algoritma");
        Iterator<String> iterator_1007 = q_1007.iterator();
        while (iterator_1007.hasNext()) {
            System.out.print(iterator_1007.next() + " ");
        }
    }

}
```

Program pada class IterasiQueue_2511531007 digunakan untuk menampilkan isi queue menggunakan iterator. Pada awalnya dibuat sebuah queue q_1007 dengan tipe data String menggunakan LinkedList, lalu dimasukkan beberapa data seperti “Praktikum”, “Struktur”, “Data”, “Dan”, dan “Algoritma”. Setelah itu dibuat objek iterator_1007 untuk menelusuri isi queue.

Proses penampilan data dilakukan dengan perulangan while yang akan terus berjalan selama masih ada elemen berikutnya (hasNext()). Setiap elemen diambil menggunakan next() dan langsung ditampilkan. Dengan cara ini, semua isi queue bisa ditampilkan secara berurutan tanpa menghapus datanya.

2.2.5 Class QueueLinkedList_2511531007

```java
package pekan4_2511531007;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511531007 {

    public static void main(String[] args) {
        Queue<Integer> q_1007 = new LinkedList<>();
        // tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
        for (int i = 0; i < 6; i++) {
            q_1007.add((i));
        }
        // menampilkan isi antrian.
        System.out.println("Elemen Antrian " + q_1007);
        // untuk menghapus kepala antrian.
        int hapus_1007 = q_1007.remove();
        System.out.println("Hapus elemen = " + hapus_1007);
        System.out.println(q_1007);
        // untuk melihat antrian terdepan
        int depan_1007 = q_1007.peek();
        System.out.println("Kepala Antrian = " + depan_1007);

        int banyak_1007 = q_1007.size();
        System.out.println("Size Antrian " + banyak_1007);
    }

}
```

Program pada class QueueLinkedList_2511531007 digunakan untuk menunjukkan penggunaan queue dengan bantuan LinkedList. Pertama dibuat queue q_1007, lalu ditambahkan elemen 0 sampai 5 menggunakan perulangan. Setelah itu, isi antrian ditampilkan untuk melihat semua data yang sudah masuk.

Selanjutnya dilakukan penghapusan satu elemen menggunakan remove(), yang otomatis menghapus data paling depan sesuai prinsip FIFO. Program kemudian menampilkan kembali isi antrian setelah penghapusan. Untuk melihat elemen terdepan tanpa menghapusnya, digunakan method peek(). Terakhir, size() digunakan untuk mengetahui jumlah elemen yang ada di dalam antrian saat itu.

2.3 Tugas Praktikum

2.3.1 Class AntrianLoket_2511531007

```java
package tugasPraktikum_pekan4_2511531007;

import java.util.Scanner;

class QueueArray_2511531007 {
    int front_1007, rear_1007, max_1007;
    String queue_1007[];

    public QueueArray_2511531007(int max_1007) {
        this.max_1007 = max_1007;
        queue_1007 = new String[max_1007];
        front_1007 = -1;
        rear_1007 = -1;
    }

    boolean isEmpty_1007() {
        return rear_1007 == -1;
    }

    boolean isFull_1007() {
        return rear_1007 == max_1007 - 1;
    }

    void enqueue_1007(String data_1007) {
        if (isFull_1007()) {
            System.out.println("Antrian penuh!");
            return;
        }
        if (isEmpty_1007()) {
            front_1007 = 0;
        }
        rear_1007++;
        queue_1007[rear_1007] = data_1007;
        System.out.println("Data berhasil ditambahkan ke antrian");
    }

    void dequeue_1007() {
        if (isEmpty_1007()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println(queue_1007[front_1007] + " telah dilayani");

        if (front_1007 == rear_1007) {
            front_1007 = -1;
            rear_1007 = -1;
        } else {
            front_1007++;
        }
    }

    void display_1007() {
        if (isEmpty_1007()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("Isi antrian:");
        int no_1007 = 1;
        for (int i_1007 = front_1007; i_1007 <= rear_1007; i_1007++) {
            System.out.println(no_1007 + ". " + queue_1007[i_1007]);
            no_1007++;
        }
    }
```

```java
    void reverse_1007() {
        if (isEmpty_1007()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("Isi antrian:");
        int start_1007 = front_1007;
        int end_1007 = rear_1007;
        while (start_1007 < end_1007) {
            String temp_1007 = queue_1007[start_1007];
            queue_1007[start_1007] = queue_1007[end_1007];
            queue_1007[end_1007] = temp_1007;

            start_1007++;
            end_1007--;
        }
        int no_1007 = 1;
        for (int i_1007 = front_1007; i_1007 <= rear_1007; i_1007++) {
            System.out.println(no_1007 + ". " + queue_1007[i_1007]);
            no_1007++;
        }
    }
}

public class AntrianLoket_2511531007 {
    public static void main(String[] args) {
        Scanner input_1007 = new Scanner(System.in);
        QueueArray_2511531007 q_1007 = new QueueArray_2511531007(100);

        int choose_1007;

        System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Reverse");
        System.out.println("5. Keluar");
        do {
            System.out.print("\nPilih menu: ");
            choose_1007 = input_1007.nextInt();
            input_1007.nextLine();

            switch (choose_1007) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama_1007 = input_1007.nextLine();
                    q_1007.enqueue_1007(nama_1007);
                    break;

                case 2:
                    q_1007.dequeue_1007();
                    break;

                case 3:
                    q_1007.display_1007();
                    break;

                case 4:
                    q_1007.reverse_1007();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid");
            }

        } while (choose_1007 != 5);

        input_1007.close();
    }
}
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

Pilih menu: 4
Isi antrian:
1. Budi
2. Andi

Pilih menu: 2
Budi telah dilayani

Pilih menu: 3
Isi antrian:
1. Andi

Pilih menu: 5
Program selesai
```

Class AntrianLoket_2511531007 berfungsi sebagai program utama yang mengatur jalannya aplikasi antrian. Di dalam class ini dibuat objek dari class QueueArray_2511531007 yang digunakan untuk menjalankan semua operasi queue. Selain itu, class ini juga menangani interaksi dengan pengguna melalui menu, seperti menambah antrian, menghapus antrian, menampilkan isi antrian, dan melakukan reverse. Alur program dikendalikan menggunakan perulangan sehingga menu akan terus muncul sampai pengguna memilih keluar. Dengan kata lain, class ini menjadi penghubung antara pengguna dengan sistem queue yang sudah dibuat.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum dan tugas yang telah dilakukan, dapat disimpulkan bahwa queue merupakan struktur data yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi yang pertama keluar. Queue memiliki beberapa operasi dasar seperti enqueue, dequeue, dan display yang digunakan untuk mengelola data dalam antrian.

Implementasi queue dalam Java dapat dilakukan dengan menggunakan library seperti LinkedList maupun secara manual menggunakan array. Penggunaan library mempermudah proses pemrograman, sedangkan implementasi dengan array membantu dalam memahami cara kerja queue secara lebih mendalam. Dengan memahami kedua cara tersebut, mahasiswa diharapkan mampu menerapkan konsep queue secara tepat dalam berbagai kasus yang membutuhkan pengelolaan data secara terurut.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Diharapkan asisten praktikum dapat memberikan evaluasi atau umpan balik terhadap laporan maupun program yang telah dibuat, sehingga mahasiswa dapat mengetahui letak kesalahan dan melakukan perbaikan.
4. Dengan adanya latihan yang cukup dan evaluasi yang jelas, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

[1] UPI Y.A.I., "Pertemuan 07 – Queue (Antrian)," Materi Struktur Data dan Algoritma, 2023. [Online]. Available: https://dosen.upi-yai.ac.id/v5/dokumen/materi/011061/73_20230506060924_Pert%2007%20-%20Queue.pdf [Diakses: 29 April 2026].